Machine Learning

Final project presentation

Izabela Kosiec 441755

Katarzyna Pełka 432453

Regression

Models taken into consideration:

- Ridge
- Lasso
- Elastic Net
- SVR
- Random Forest Regressor
- Linear Regression
- XGBRegressor - winner

Step by step:

1. Loading data & libraries
2. Initial data analysis
3. Data cleaning
4. Feature engineering
5. Feature selection
6. Preprocessing pipeline
7. Training & tuning
8. Model evaluation
9. Final prediction

Libraries used

```
#necessary libraries
!pip install xgboost
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV, KFold
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV
from sklearn.svm import LinearSVR
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVR
from sklearn.linear_model import Ridge, Lasso, ElasticNet, LinearRegression
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
```

Handling missing values

```
appartments[columns_with_missings].isnull().sum()
```

```
obj_type                 33820
floor_no                 27698
floor_max                 1921
year_built               25713
dist_sch                   130
dist_clinic                571
dist_post                  188
dist_kind                  163
dist_rest                  391
dist_uni                  4317
dist_pharma                219
build_mat                61999
cond_class              117058
has_lift                  7727
infrastructure_quality   25713
dtype: int64
```

- ***year_built*** - filling NAs with median, adding additional variable: year_built_missing (yes/no)
- for all ***distances*** - replace missing values with median
- ***obj_type*** and ***build_mat*** - filling missing values with missing - as another category in this variable
- ***has_lift*** - if observation with missing value has floor_max >4 assign “yes”, otherwise “no”
- ***cond_class*** **-** more than 117 000 observations with missing values, removed from dataset
- ***infrastucture_quality*** - using the mean infrastructure quality for the specific year_build for that observation

! We always calculate on train data and based on that we fill out missing values in test data !

Handling missing values

```
most_common_floor_no = appartments['floor_no'].mode()[0]
def impute_floor_no(row, most_common):
    if pd.isna(row['floor_no']):
        if not pd.isna(row['floor_max']):
            return max(round(row['floor_max'] / 2), 1.0)
        elif row['has_lift'] == 'yes':
            return 5.0
        else:
            return most_common
    else:
        return row['floor_no']
```

- ***floor_no*** - filling NAs with half of the floor number from the floor_max or floor_no 5 if there is lift. In other cases, assigns mode

- ***floor_max*** - filling NAs based on floor_no

```
floor_max_median_train = appartments['floor_max'].median()
#  floor_max based on floor_no
def impute_floor_max(row, median_val):
    if pd.isna(row['floor_max']):
        if not pd.isna(row['floor_no']):
            if 1 <= row['floor_no'] <= 4:
                return 4
            elif 5 <= row['floor_no'] <= 10:
                return 10
            elif row['floor_no'] > 10:
                return row['floor_no']
        else:
            return median_val  # fallback if floor_no is missing too
    return row['floor_max']
```

! We always calculate on train data and based on that we fill out missing values in test data !

Variables transformation

src_month was converted to datetime and
new variables: *year, month_numer, quarter*
were created

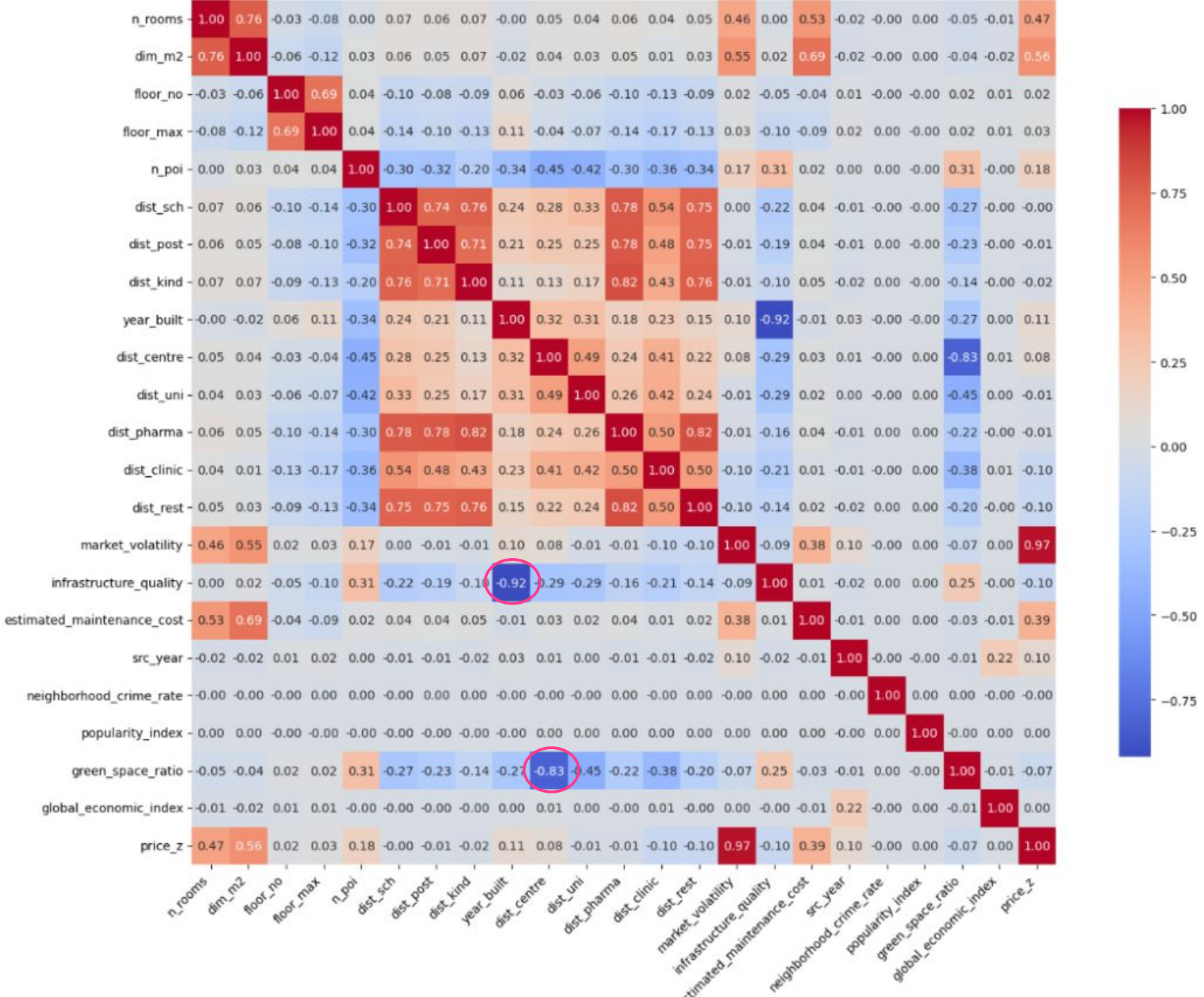
```
## Convert src_month to datetime and create variables: year, month number, quarter
for df in [appartments, test_df]:
    df['src_month'] = pd.to_datetime(df['src_month'], format="%Y-%m", errors='coerce')
    df['src_year'] = df['src_month'].dt.year
    df['src_month_num'] = df['src_month'].dt.month
    df['src_quarter_str'] = df['src_month_num'].apply(
        lambda m: '1st quarter' if m in [1, 2, 3]
        else '2nd quarter' if m in [4, 5, 6]
        else '3rd quarter' if m in [7, 8, 9]
        else '4th quarter'
    )

print(appartments['src_quarter_str'].value_counts())
print(appartments['src_year'].value_counts())
```

```
src_quarter_str
2nd quarter    48838
4th quarter    39708
1st quarter    39268
3rd quarter    28640
Name: count, dtype: int64
src_year
2024    88106
2023    68348
Name: count, dtype: int64
```

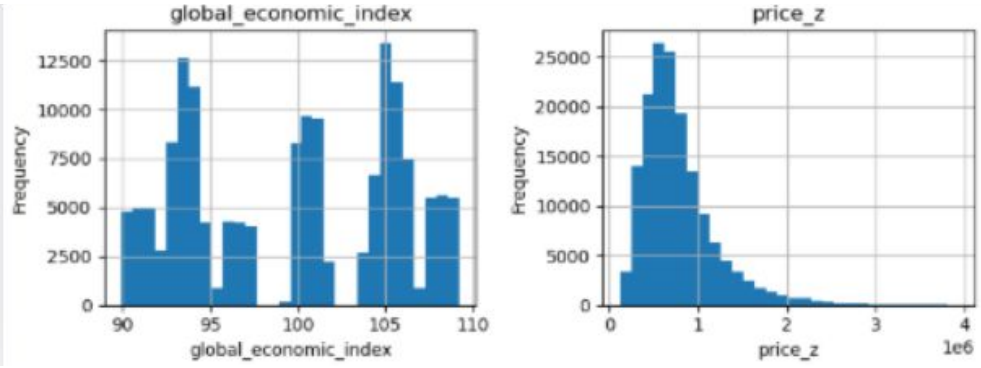
Correlation matrix of numeric variables

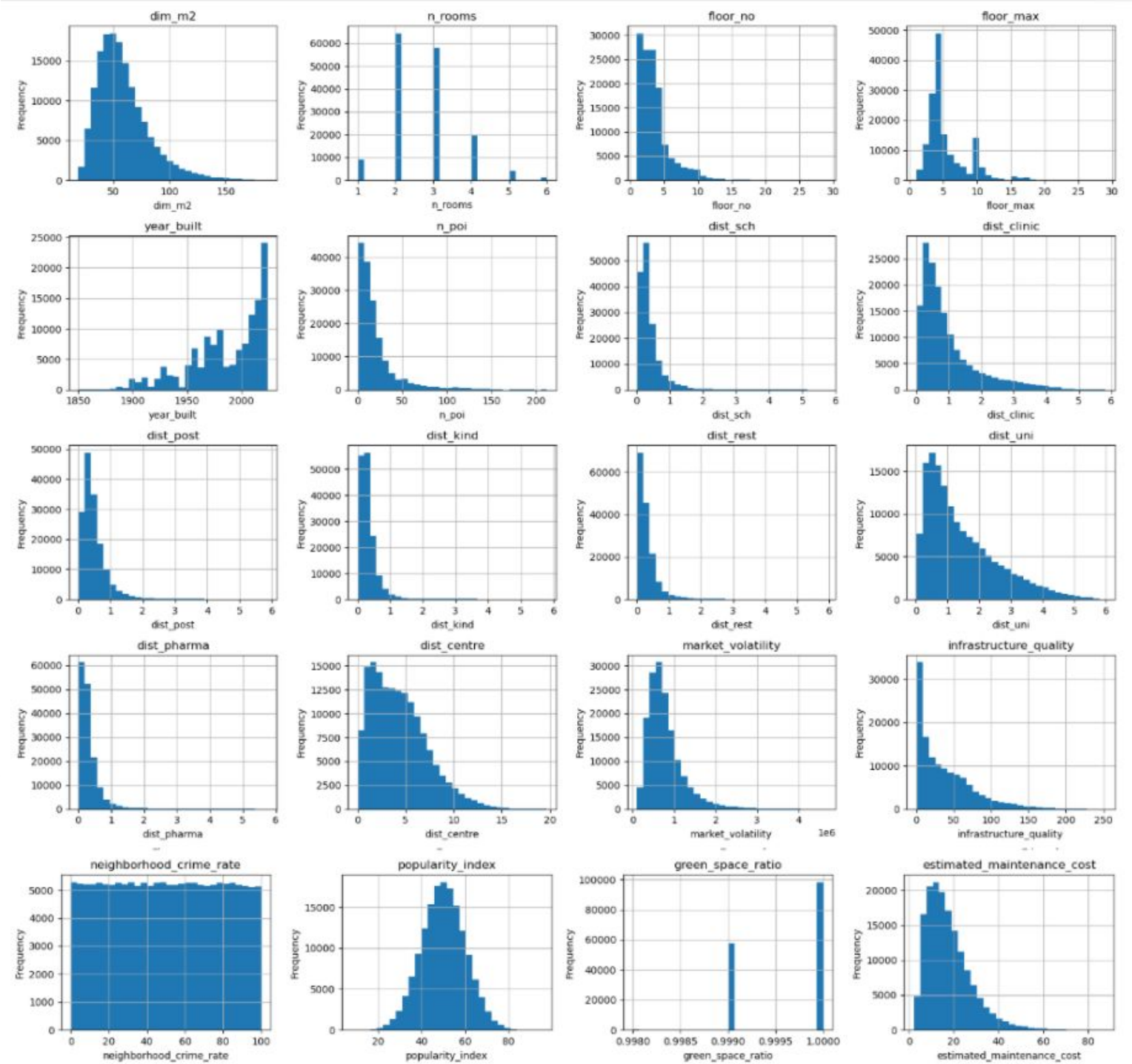
Excluding *green_space_ratio* and *infrastructure_quality* (in train and test dataset) due to high correlation

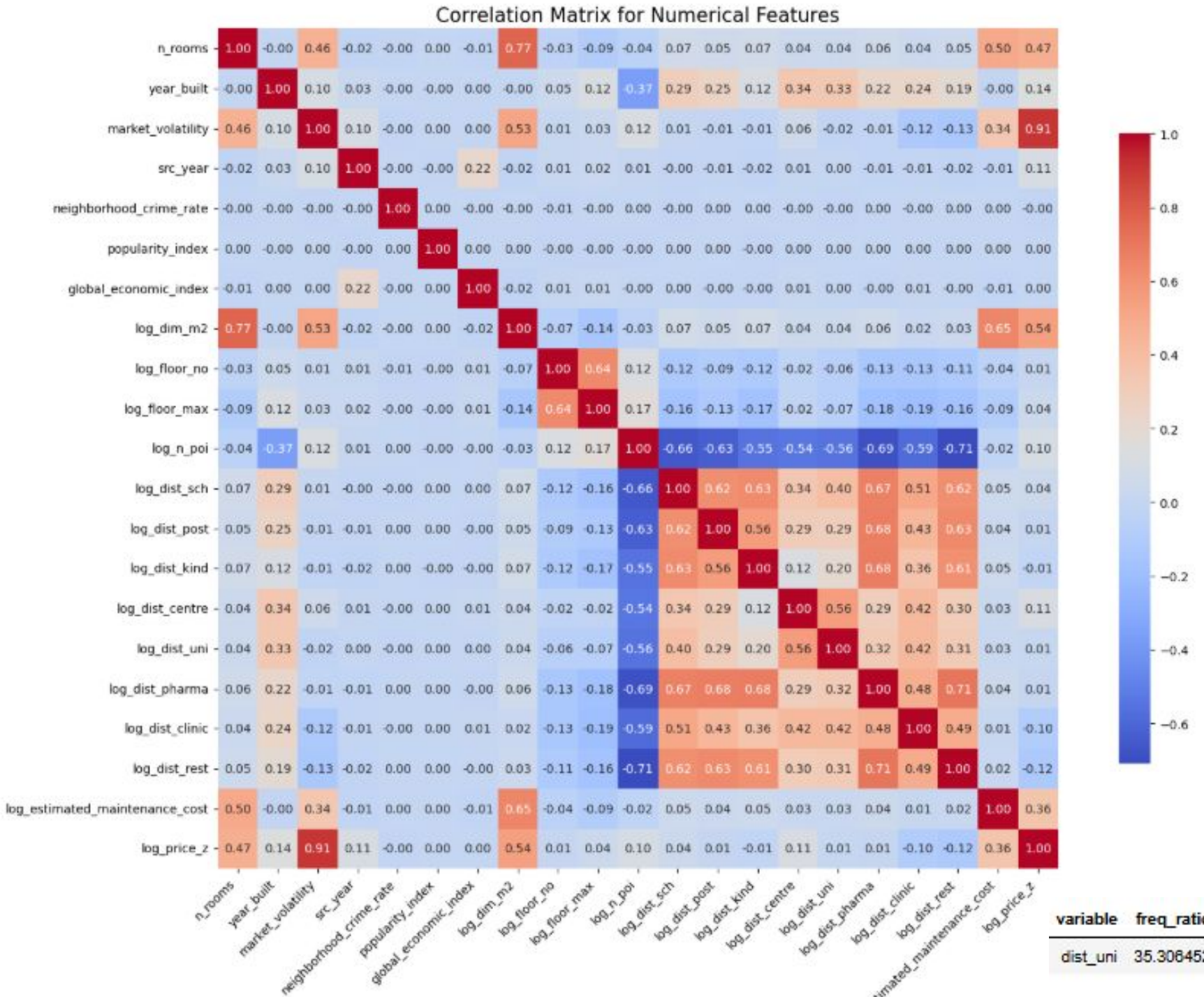


Distribution of the variables

We decided to log-transform right-skewed columns: *dim_m2, floor_no, floor_max, n_poi, dist_sch, dist_post, dist_kind, dist_rest, dist_centre, dist_uni, dist_pharma, dist_clinic, estimated_maintenance_cost*, and dependent variable ***price_z***







Correlation matrix of numeric variables

After log transformation

Near-zero variance

Only *dist_uni* had high frequency ratio and low unique ratio, but it was kept in the model due to its potential influence on ***price_z***

variable	freq_ratio	unique_ratio	high_freq_ratio	low_unique_ratio
dist_uni	35.306452	0.035397	1	1

Final variables:

```
numerical_cols1 = ['n_rooms', 'year_built','market_volatility',
                   'src_year','neighborhood_crime_rate','popularity_index', 'global_economic_index',
                   'log_dim_m2', 'log_floor_no', 'log_floor_max', 'log_n_poi', 'log_dist_sch', 'log_dist_post', 'log_dist_kind',
                    'log_dist_centre', 'log_dist_uni', 'log_dist_pharma', 'log_dist_clinic', 'log_dist_rest',
                     'log_estimated_maintenance_cost']

categorical_cols = ['obj_type', 'own_type', 'build_mat', 'has_park','has_balcony', 'has_lift', 'has_sec', 'has_store',
                     'loc_code','year_built_missing', 'src_quarter_str']

drop_cols = ['unit_id','price_z', 'log_price_z', 'dim_m2', 'floor_no', 'floor_max',  'n_poi', 'dist_sch', 'green_space_ratio',
                              'dist_post','dist_kind', 'dist_centre',  'dist_uni', 'dist_pharma','dist_clinic', 'dist_rest',
                               'infrastructure_quality',  'estimated_maintenance_cost']

X = appartments.drop(columns=drop_cols)
y = appartments['log_price_z']

numerical_cols1   = [c for c in numerical_cols1   if c in X.columns]
categorical_cols  = [c for c in categorical_cols  if c in X.columns]

missing_cols = set(X.columns) - set(test_df.columns)
if missing_cols:
    print("Missing columns in test_df:", missing_cols)
else:
    print("All columns from X are present in test_df.")
```

! All columns presented in X were present in test_df also !

Pipeline for numerical (standardization) and categorical (one-hot encoding) variables, splitting data (train and validation data)

```
num_tf = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scale',   StandardScaler())
])

cat_tf = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('oh',      OneHotEncoder(handle_unknown='ignore', sparse_output=True))
])

preprocessor = ColumnTransformer([
    ('num', num_tf, numerical_cols1),
    ('cat', cat_tf, categorical_cols)
])


X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Choosing the best models for further evaluation based on RMSE value on 10000 observations sample

Models taken into consideration:

- Ridge
- Lasso
- Elastic Net
- Linear Regression
- SVR
- Random Forest Regressor
- XGB(Regressor) - winner

RandomizedSearchCV for XGB, SVR and Elastic Net, Grid for the rest. cv=5 for all

```
if name in ["xgb", "svr"]:
    search = RandomizedSearchCV(
        pipe,
        param_distributions=param_grid_renamed,
        n_iter=40,
        cv=5,
        scoring=rmse_pln_scorer,
        n_jobs=-1,
        verbose=1,
        random_state=42
    )
else:
    search = GridSearchCV(
        pipe,
        param_grid=param_grid_renamed,
        cv=5,
        scoring=rmse_pln_scorer,
        n_jobs=-1,
        verbose=1
    )

search.fit(X_samp, y_samp)
best_models[name] = search.best_estimator_
```

RIDGE and LASSO

```
#  Grid Search for RIDGE
pipe = make_pipeline(preprocessor, Ridge(random_state=42))
param_grid = {
    "ridge__alpha": np.logspace(-3, 4, 50)
}

search = GridSearchCV(
    pipe,
    param_grid=param_grid,
    cv=5,
    scoring=rmse_pln_scorer,
    n_jobs=-1,
    verbose=1
)
search.fit(X_samp, y_samp)

best_ridge = search.best_estimator_
print("RIDGE best RMSE_pln =", abs(search.best_score_))
print("Best params:", search.best_params_)
```

```
Fitting 5 folds for each of 50 candidates, totalling 250 fits
RIDGE best RMSE_pln = 175802.8619203151
Best params: {'ridge__alpha': 7196.856730011514}
```

RIDGE:

RMSE = 176k

LASSO:

RMSE = 208k

```
# Grid Search for LASSO
pipe = make_pipeline(preprocessor, Lasso(max_iter=20000, random_state=42))
param_grid = {
    "lasso__alpha": np.logspace(-4, 0, 50)
}

search = GridSearchCV(
    pipe,
    param_grid=param_grid,
    cv=5,
    scoring=rmse_pln_scorer,
    n_jobs=-1,
    verbose=1
)
search.fit(X_samp, y_samp)

best_lasso = search.best_estimator_
print("LASSO best RMSE_pln =", abs(search.best_score_))
print("Best params:", search.best_params_)
```

```
Fitting 5 folds for each of 50 candidates, totalling 250 fits
LASSO best RMSE_pln = 207665.59850051586
Best params: {'lasso__alpha': 0.15264179671752318}
```

ELASTIC NET and LINEAR REGRESSION

```
# Randomized Search for ELASTICNET
pipe = make_pipeline(preprocessor, ElasticNet(max_iter=20000, random_state=42))
param_dist = {
    "elasticnet__alpha": uniform(0.0001, 1),
    "elasticnet__l1_ratio": uniform(0.1, 0.9)
}

search = RandomizedSearchCV(
    pipe,
    param_distributions=param_dist,
    n_iter=30,
    cv=5,
    scoring=rmse_pln_scorer,
    n_jobs=-1,
    verbose=1,
    random_state=42
)
search.fit(X_samp, y_samp)

best_elastic = search.best_estimator_
print("ELASTICNET best RMSE_pln =", abs(search.best_score_))
print("Best params:", search.best_params_)
```

```
Fitting 5 folds for each of 30 candidates, totalling 150 fits
ELASTICNET best RMSE_pln = 201917.81207718846
Best params: {'elasticnet__alpha': 0.3047137691733707, 'elasticnet__l1_ratio': 0.1879049026057455}
```

ELASTIC NET:

RMSE = 202k

LINEAR REGRESSION:

RMSE = 257k

```
#Grid Search for LINEAR REGRESSION
pipe = make_pipeline(preprocessor, LinearRegression())
param_grid = {}  #no hyperparameters
search = GridSearchCV(
    pipe,
    param_grid=param_grid,
    cv=5,
    scoring=rmse_pln_scorer,
    n_jobs=-1,
    verbose=1
)
search.fit(X_samp, y_samp)
best_linreg = search.best_estimator_
print("LINREG best RMSE_pln =", abs(search.best_score_))
```

```
Fitting 5 folds for each of 1 candidates, totalling 5 fits
LINREG best RMSE_pln = 257396.15020578052
```

SVR: RMSE = 112k

```
#RandomizedSearchCV for SVR
pipe = make_pipeline(preprocessor, SVR())
param_dist = {
    "svr__C":        [0.01, 0.1, 1, 10, 100, 500],
    "svr__epsilon":  [0.001, 0.01, 0.1, 0.5, 1],
    "svr__kernel":   ["rbf", "poly", "sigmoid"],
    "svr__gamma":    ["scale", "auto", 0.01, 0.1, 1]
}
search = RandomizedSearchCV(
    pipe,
    param_distributions=param_dist,
    n_iter=30,
    cv=5,
    scoring=rmse_pln_scorer,
    n_jobs=-1,
    verbose=1,
    random_state=42
)
search.fit(X_samp, y_samp)
best_svr = search.best_estimator_
print("SVR best RMSE_pln =", abs(search.best_score_))
print("Best params:", search.best_params_)
```

```
SVR best RMSE_pln = 111,588
Best params = {'svr__kernel': 'rbf', 'svr__gamma': 0.01, 'svr__epsilon': 0.001, 'svr__C': 10}
```

XGBoost : RMSE = 96k

```
# Grid Search for XGBOOST
pipe = make_pipeline(preprocessor, XGBRegressor(
    tree_method="hist",
    objective="reg:squarederror",
    eval_metric="rmse",
    n_jobs=-1,
    random_state=42
))
param_grid = {
    "xgbregressor__n_estimators": [300, 400, 500],
    "xgbregressor__max_depth": [3,4,5,6],
    "xgbregressor__learning_rate": [0.02, 0.03, 0.05],
    "xgbregressor__subsample": [0.7, 0.8, 1.0],
    "xgbregressor__colsample_bytree": [0.7, 0.8, 1.0]
}

search = GridSearchCV(
    pipe,
    param_grid=param_grid,
    cv=5,
    scoring=rmse_pln_scorer,
    n_jobs=-1,
    verbose=1
)
search.fit(X_samp, y_samp)
best_xgb = search.best_estimator_
print("XGB best RMSE_pln =", abs(search.best_score_))
print("Best params:", search.best_params_)
```

XGB best RMSE_pln = 95743.25

Best params:

- 'xgbregressor__colsample_bytree': 0.8,
- 'xgbregressor__learning_rate': 0.03,
- 'xgbregressor__max_depth': 4,
- 'xgbregressor__n_estimators': 500,
- 'xgbregressor__subsample': 0.7

Random Forest : RMSE = 97k

```
#  Grid Search for RANDOM FOREST
pipe = make_pipeline(preprocessor, RandomForestRegressor(n_estimators=300, n_jobs=-1, random_state=42))
param_grid = {
    "randomforestregressor__n_estimators": [200,300],
    "randomforestregressor__max_depth": [10,15,20],
    "randomforestregressor__max_features": [ 0.6, 1.0],
    "randomforestregressor__min_samples_split": [2, 5],
    "randomforestregressor__min_samples_leaf": [1, 2]
}
search = GridSearchCV(
    pipe,
    param_grid=param_grid,
    cv=5,
    scoring=rmse_pln_scorer,
    n_jobs=-1,
    verbose=1
)
search.fit(X_samp, y_samp)
best_rf = search.best_estimator_
print("RF best RMSE_pln =", abs(search.best_score_))
print("Best params:", search.best_params_)
```

RF best RMSE_pln = 97334.46

Best params: {'randomforestregressor__max_depth': 20, 'randomforestregressor__max_features': 0.6, 'randomforestregressor__min_samples_leaf': 1, 'randomforestregressor__min_samples_split': 2, 'randomforestregressor__n_estimators': 300}

Hyper Parameters tuning - tested combinations

Due to their strong performance on a reduced 10k sample, XGBoost and Random Forest were selected for final model development. Extensive hyperparameter tuning was carried out for both models, followed by 5-fold cross-validation on the training set to evaluate performance. The best-performing configurations were then assessed on both training and validation sets to compare RMSE values and detect potential overfitting.

Parameter ranges tested for each test

Random Forest	The best Random forest	XGBoost	The best XGBoost
max_depth = [15,20,23,25]	**max_depth** = [23]	**max_depth** = [4,5,6,7, 8]	**max_depth** = [8]
max_features = [0.6,0.7]	**max_features** = [0.7]	**learning_rate** = [0.01, 0.02,0.03]	**learning_rate** = [0.02]
n_estimators = [200,250,300]	**n_estimators** = [250]	**n_estimators** = [500,590,600]	**n_estimators** = [600]
min_samples_split =[2,3]	**min_samples_split** =[3]	**subsample** = [0.65, 0.7,0.8]	**subsample** = [0.7]
min_samples_leaf = [1,2]	**min_samples_leaf** = [1]	**colsample_bytree** = [0.8, 0.85,0.90]	**colsample_bytree** = [0.8]

Final decision

The XGBoost model achieved an RMSE of 81,921 PLN on the training set and 95,074 PLN on the validation set, indicating good generalization with limited overfitting. In contrast, Random Forest obtained a much lower training RMSE of 38,384 PLN but a similar validation RMSE of 96,014 PLN, revealing signs of overfitting. Therefore, XGBoost was ultimately selected as the final model due to its more balanced performance and greater robustness in predicting unseen data.

Best Parameters: {'xgbregressor__subsample': 0.7, 'xgbregressor__n_estimators': 600, 'xgbregressor__max_depth': 8, 'xgbregressor__learning_rate': 0.02, 'xgbregressor__colsample_bytree': 0.8}

```python
#final XGB
best_xgb = XGBRegressor(
    tree_method="hist",
    objective="reg:squarederror",
    eval_metric="rmse",
    n_estimators=600,
    max_depth=8,
    learning_rate=0.02,
    subsample=0.7,
    colsample_bytree=0.8,
    n_jobs=-1,
    random_state=42
)
final_pipe = make_pipeline(preprocessor, best_xgb)
final_pipe.fit(X, y)

missing_cols = set(X.columns) - set(test_df.columns)
for col in missing_cols:
    test_df[col] = np.nan

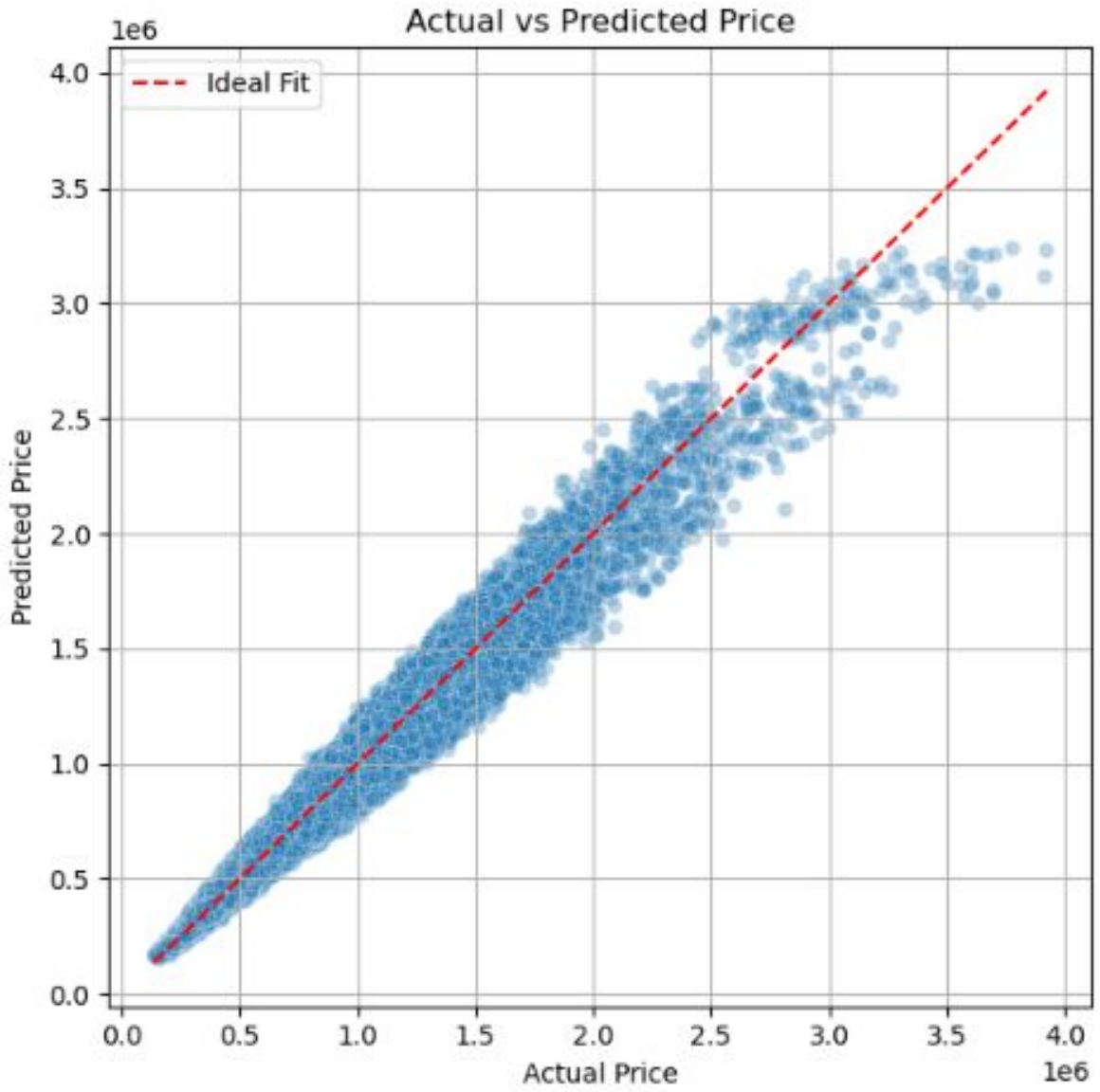
X_test = test_df[X.columns]
log_preds = final_pipe.predict(X_test)
price_preds = np.expm1(log_preds)

os.makedirs("final", exist_ok=True)
pred_df = test_df[['unit_id']].copy()
pred_df['price_z_pred'] = price_preds
pred_df.to_csv("final/final_xgb_predictions_Kosiec_Pełka.csv", index=False)
print("Predictions saved to final/final_xgb_predictions_Kosiec_Pełka.csv")
```

Final XGB model

The final XGBoost model uses the best hyperparameters selected through tuning, including 600 trees, a learning rate of 0.02, and maximum depth of 8. It is trained on the full dataset and applied to the test set using a consistent preprocessing pipeline. The predicted prices are transformed back from logarithmic scale and saved as the final output.

Actual vs Predicted Price (on validation set)

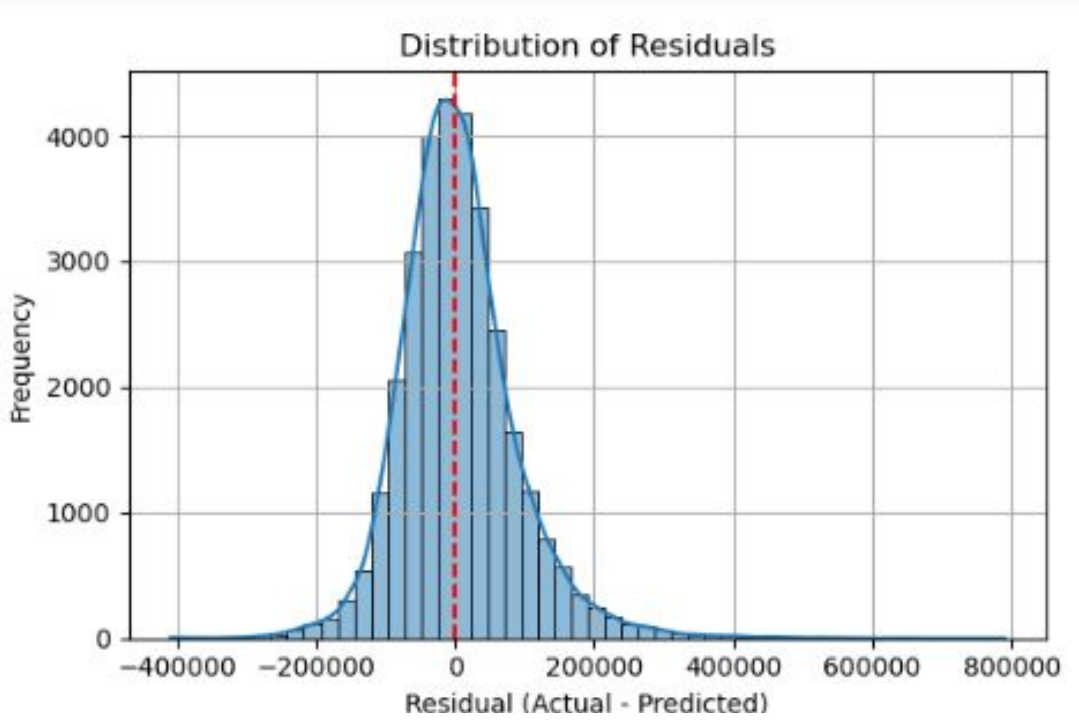


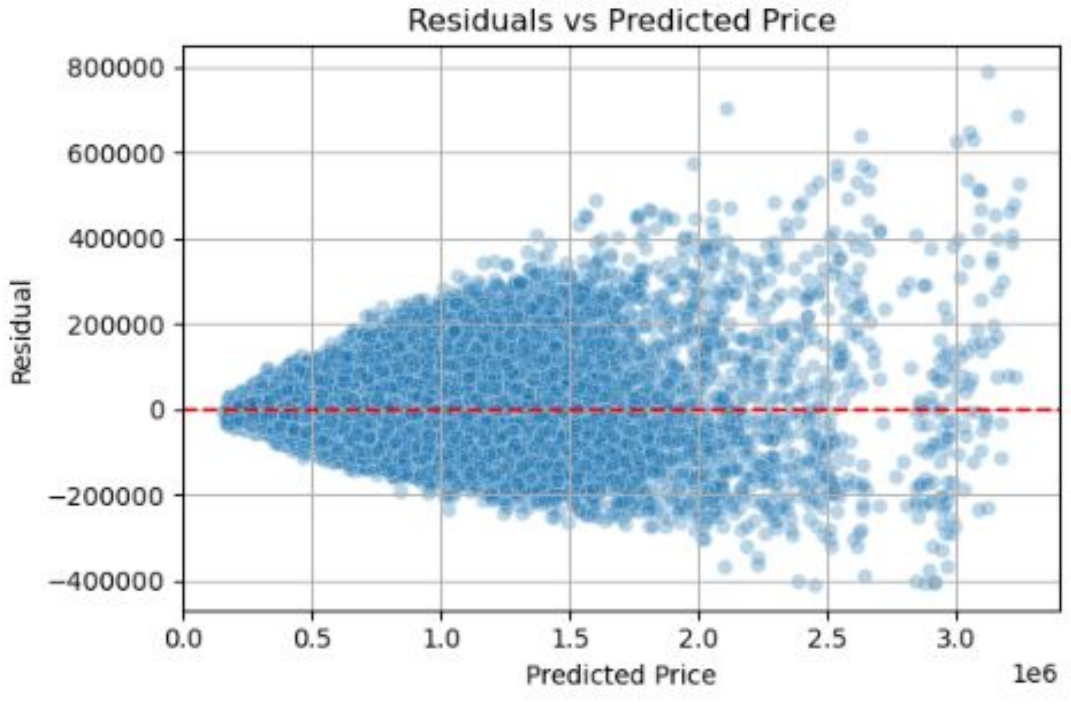
The XGBoost model fits the data well and predicts housing prices with high accuracy.

For very expensive properties, prediction accuracy decreases slightly - which is typical and may result from fewer observations in that range or stronger nonlinear effects.

Residuals

The residual analysis confirms that the XGBoost model performs well, with errors mostly centered around zero and a roughly normal distribution. Residuals remain small and symmetric across most predicted prices, although their spread increases for higher-value properties. This indicates solid overall model fit with slightly reduced precision at the upper end of the price range.





Classification

Models taken into consideration:

- Logistic regression
- KNN
- SVM
- XGBoost

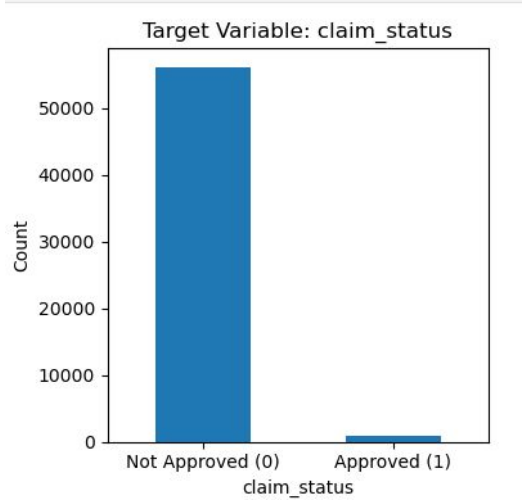
Step by step:

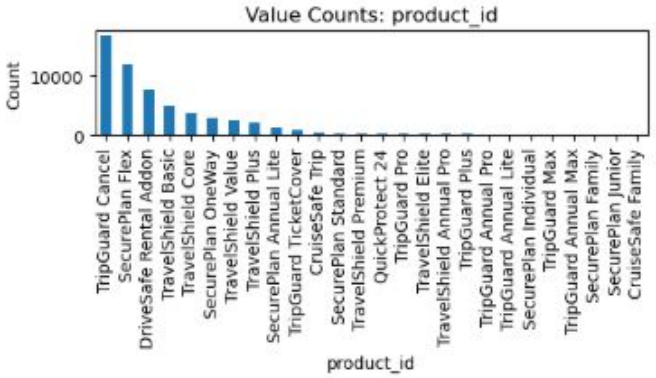
1. Loading data & libraries
2. Initial data analysis
3. Data cleaning
4. Feature engineering
5. Feature selection
6. Preprocessing pipeline
7. Training & tuning
8. Model evaluation
9. Final prediction

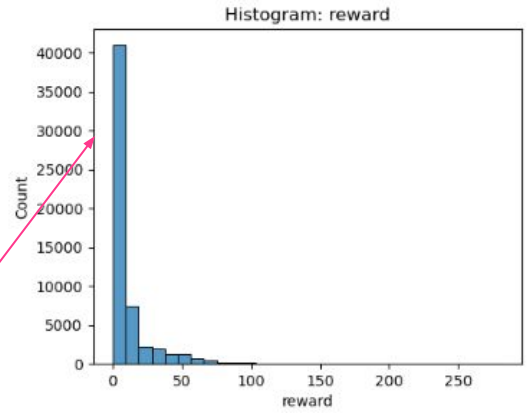
Libraries used

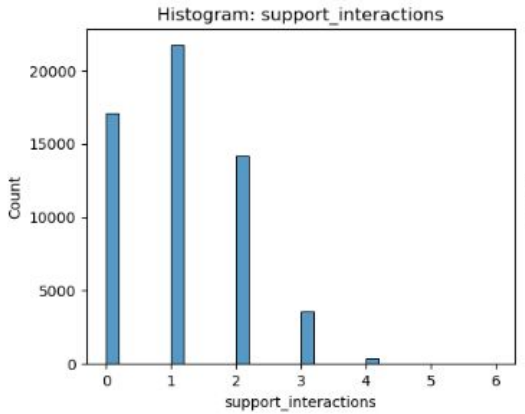
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from xgboost import XGBClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import balanced_accuracy_score
import pickle
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import RandomOverSampler, SMOTE, ADASYN
from imblearn.combine import SMOTETomek
from imblearn.pipeline import Pipeline
from imblearn.under_sampling import RandomUnderSampler
from imblearn.under_sampling import TomekLinks
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.metrics import make_scorer
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import cross_validate
from sklearn.metrics import recall_score, precision_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
from sklearn.metrics import accuracy_score, precision_score, recall_score
```

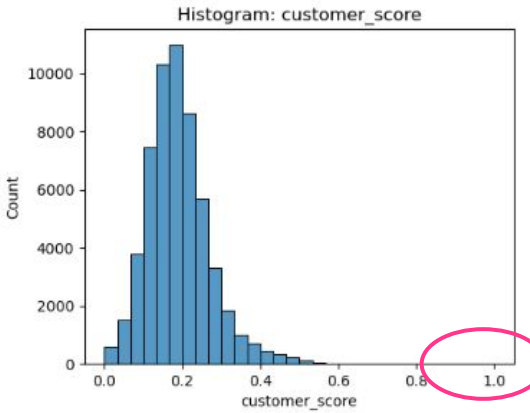
Distribution of the variables

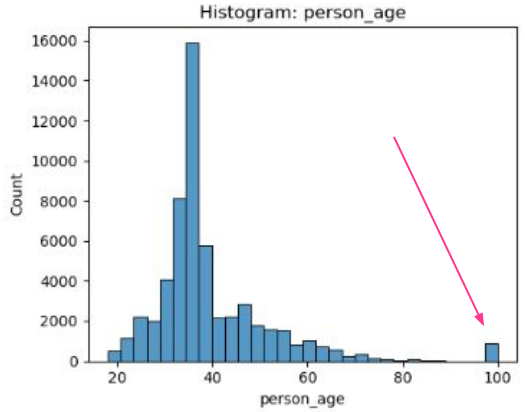


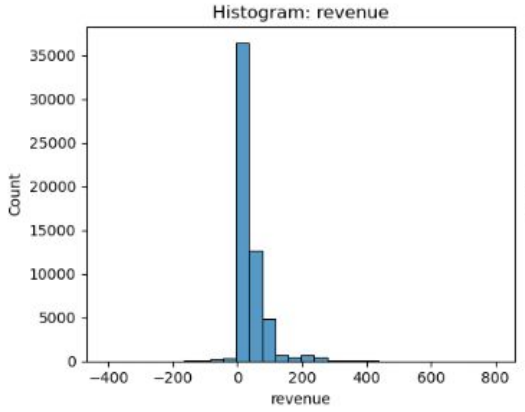


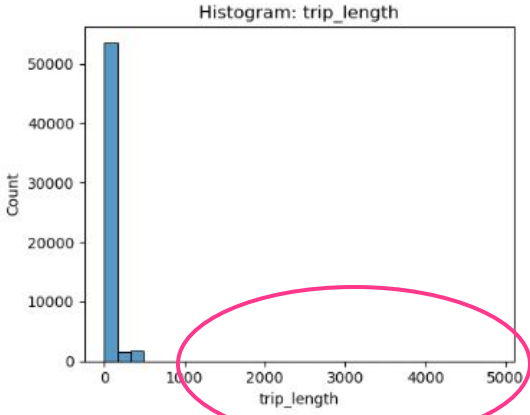


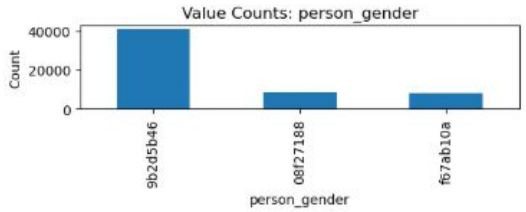


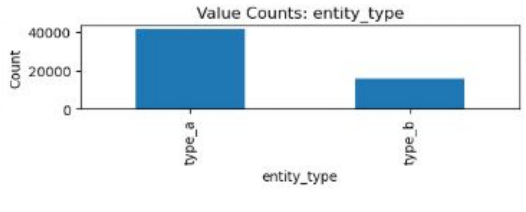


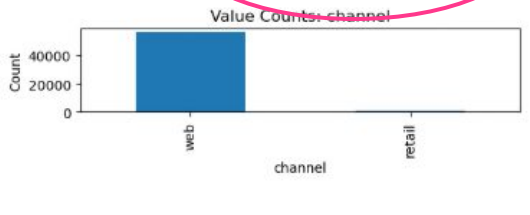


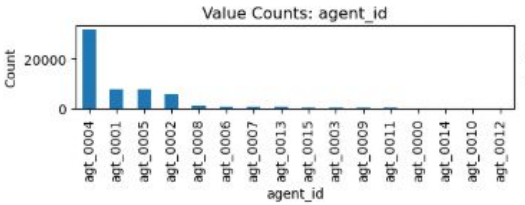


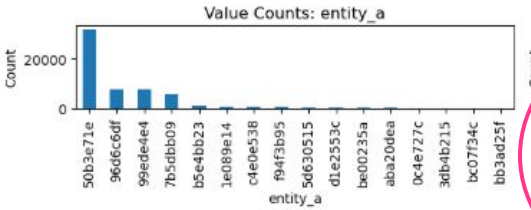


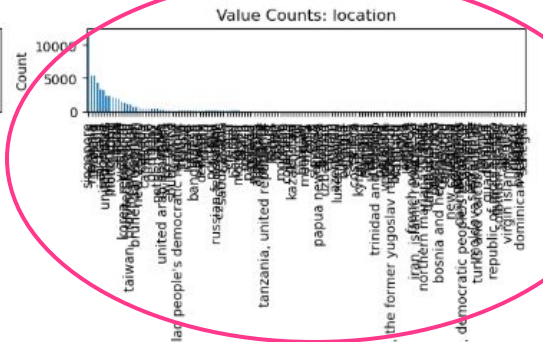












Descriptive statistics and transformation of variables

Descriptive Statistics for Numerical Variables

	count	mean	std	min	25%	50%	75%	max
reward	56992.0	9.790199	19.758401	0.000000	0.000000	0.000000	11.336448	273.752509
claim_status	56992.0	0.014634	0.120082	0.000000	0.000000	0.000000	0.000000	1.000000
support_interactions	56992.0	1.093961	0.923110	0.000000	0.000000	1.000000	2.000000	6.000000
customer_score	56992.0	0.189526	0.080570	0.000000	0.136388	0.181039	0.230836	0.781538
person_age	56992.0	39.696203	12.601012	18.000000	34.000000	36.000000	44.000000	100.000000
revenue	56992.0	40.736729	48.645504	-406.642669	18.246129	26.784930	47.808126	667.746690
trip_length	56992.0	49.233313	98.659618	1.000000	9.000000	22.000000	53.000000	4856.000000

Age of 100 was considered as error in data imputation and treated as ‚age unknown', values 100 were switched to median age

Trip length above 4000 was also considered as error during the imputation, after looking closely at observations with such vales, all of them had the same agent, destination and product type, so these values were switched to median from group of the common variables across them

```
median_singapore_tripguard = train_df[
    (train_df['location'] == 'singapore') &
    (train_df['product_id'] == 'TripGuard TicketCover')
]['trip_length'].median()

def replace_trip_lengths_above_4000(df, median_value):
    df.loc[df['trip_length'] > 4000, 'trip_length'] = median_value
    return df

print(f"Mediana trip_length dla Singapore + TripGuard TicketCover: {median_singapore_tripguard}")

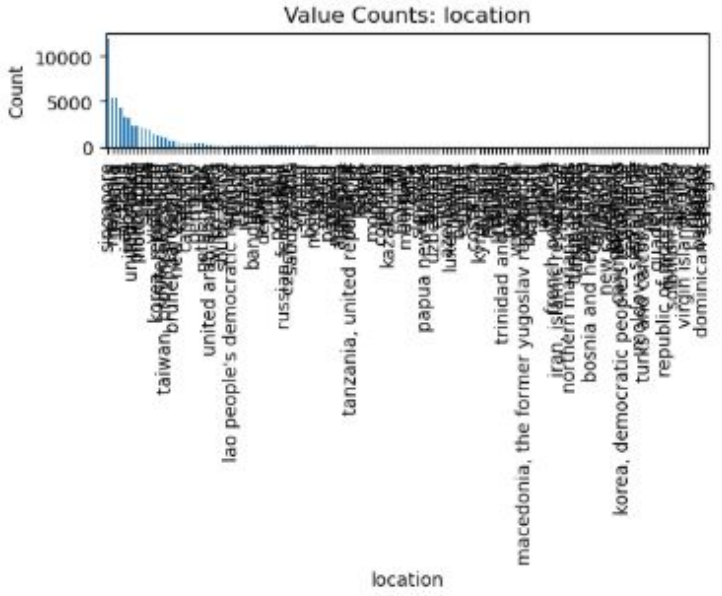
train_df = replace_trip_lengths_above_4000(train_df, median_singapore_tripguard)
test_df = replace_trip_lengths_above_4000(test_df, median_singapore_tripguard)
```

Because of high rate of „0" values in *reward,* additional variable was created to indicate if the reward was 0 or not

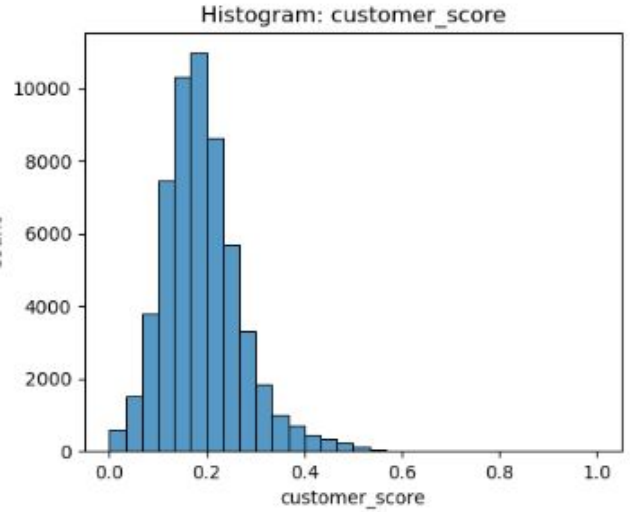
Descriptive Statistics for Categorical Variables

	count	unique	top	freq
person_gender	56992	3	9b2d5b46	40643
entity_type	56992	2	type_a	41297
channel	56992	2	web	56018
agent_id	56992	16	agt_0004	31663
entity_a	56992	16	50b3e71e	31663
location	56992	146	singapore	11942
product_id	56992	26	TripGuard Cancel	16795
reward_zero	56992	2	yes	31748

Descriptive statistics and transformation of variables



Because of multiple different values in variable location it was transformed to recode levels that count is lower than 0.5% of numer of all observation as „others”



There was also one observation with customer score = 1, as there's no 'ideal customer', it was treated as mistake in the data and observation was deleted

Similar operation was conducted for product_id, but the grouping was performer on product types

```
def group_product(prod):
    if "TripGuard" in prod:
        return "TripGuard"
    elif "SecurePlan" in prod:
        return "SecurePlan"
    elif "DriveSafe" in prod:
        return "DriveSafe"
    elif "TravelShield" in prod:
        return "TravelShield"
    elif "QuickProtect" in prod:
        return "QuickProtect"
    elif "CruiseSafe" in prod:
        return "CruiseSafe"
    else:
        return "Other"


train_df["prod_id_grouped"] = train_df["product_id"].apply(group_product)
```

```
test_df["prod_id_grouped"] = test_df["product_id"].apply(group_product)
```

```
print(train_df[["product_id", "prod_id_grouped"]].head(10))
```

Correlation matrix of numeric variables

We also decided to exclude variables *support_interactions* and *person_age* from modeling due to their lack of correlation with dependent variable.

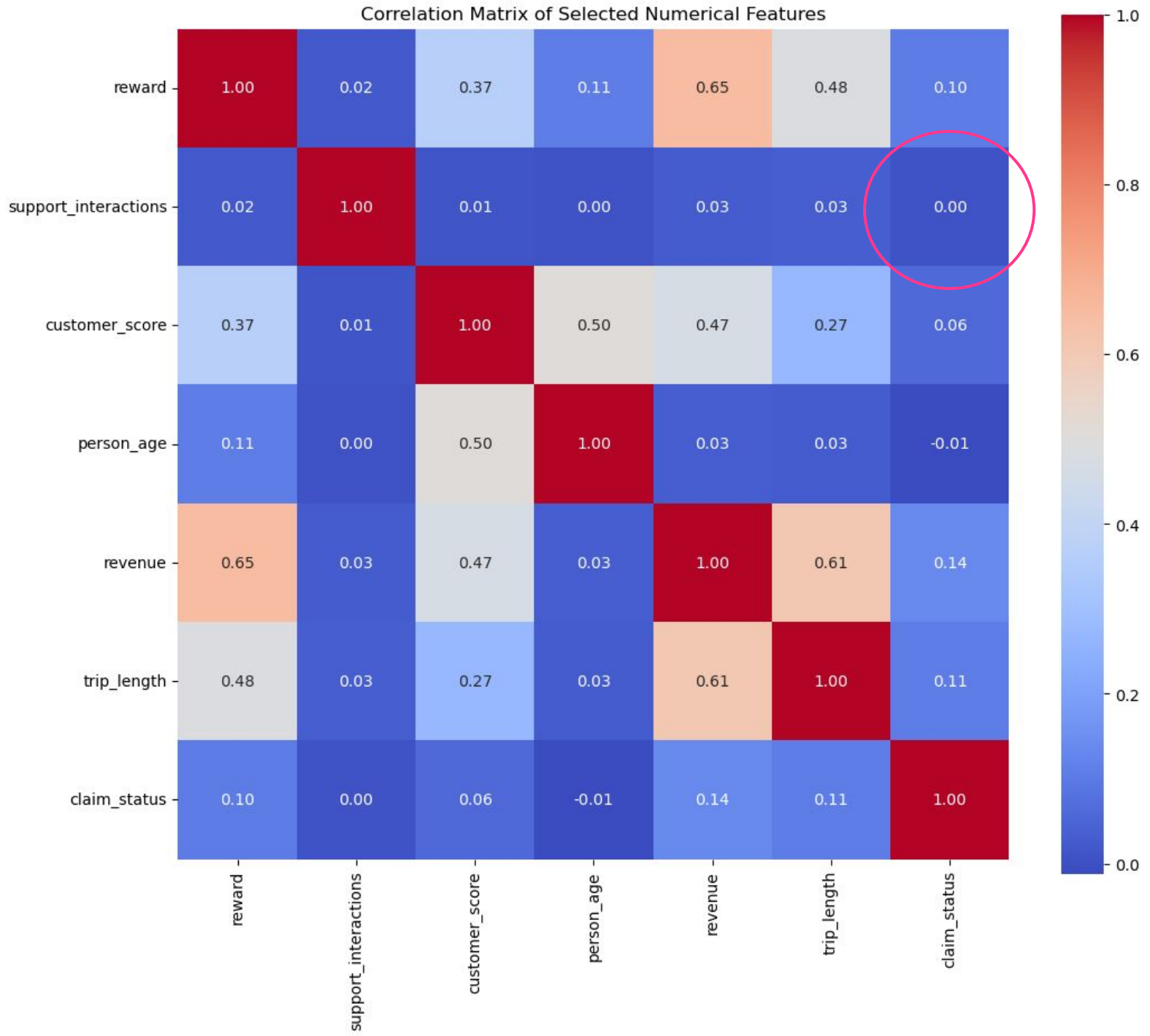
Near-zero variance

```
cat_data = train_df[categ_cols].copy()
nzv_result = near_zero_var(cat_data)
print(nzv_result)
```

```
          variable  freq_ratio  unique_ratio  high_freq_ratio  \
2          channel   57.513347      0.000035                1
0    person_gender    4.850579      0.000053                0
3         agent_id    4.105147      0.000281                0
4         entity_a    4.105147      0.000281                0
1      entity_type    2.631220      0.000035                0
5         location    2.230065      0.000456                0
7      reward_zero    1.257645      0.000035                0
6  prod_id_grouped    1.108387      0.000105                0

   low_unique_ratio
2                 1
0                 1
3                 1
4                 1
1                 1
5                 1
7                 1
```

Due to near-zero variance (high frequency ratio and low unique ratio, we decided to remove also variable *channel*



Anova results for categorical variables that left

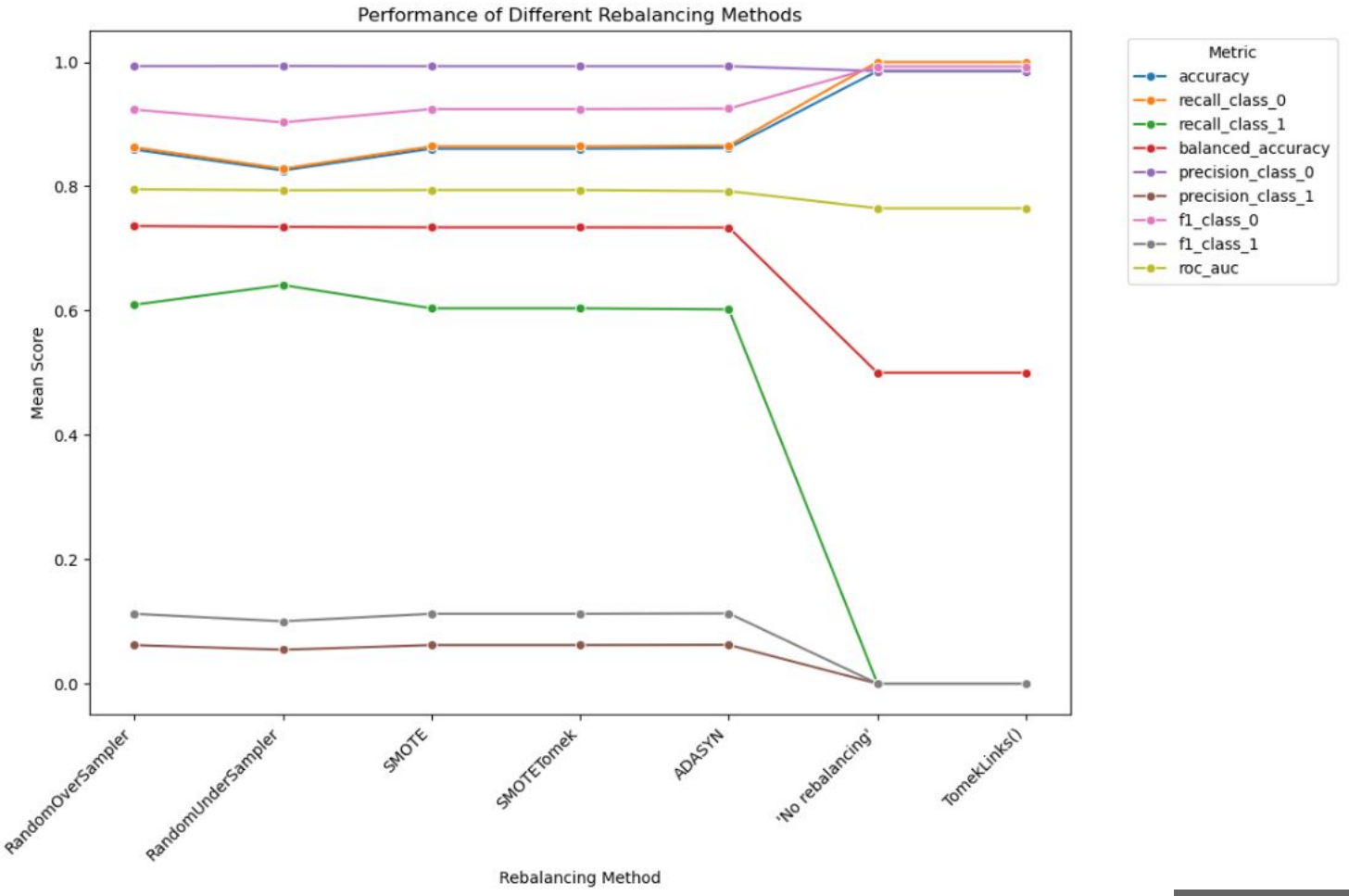
	Variable	F-statistic	p-value
1	entity_type	577.985016	4.433459e-127
6	reward_zero	410.806034	5.131077e-91
0	person_gender	355.604532	3.300957e-154
2	agent_id	116.395956	0.000000e+00
3	entity_a	116.395956	0.000000e+00
5	prod_id_grouped	72.391901	8.172697e-76
4	location	34.092580	2.166485e-162

All categorical variables that we kept in the model have significant impact on *claim_status*

Rebalancing methods tested

- Random Over Sampler
- Random Under Sampler
- SMOTE
- Tomek Links
- SMOTETomek
- ADASYN

Rebalancing



We firmly tested different rebalancing methods on Logistic regression only to have an overview of general performance. They all give similar results except of Tomek Links

Hyperparapeters tuning - tested combinations

Hyperparameters were tuned by testing combinations of multiple parameters for all models and combined with all resampling methods mentioned before. Testing was done using cross-validation with 5 folds on train test sample only (before earlier dividing train data into train and validation subsamples)

Parameter ranges tested for each test

Logistic regression	**KNN**	**SVM**	**XGBoost**
C = [0.001, 0.01, 0.1, 0.5, 1, 5, 10, 25, 50, 100, 500]	**n** = range(1, 51)	**C** = [0.01, 0.05, 0.1, 0.5, 1, 5, 10, 25, 50, 100]	**max_depth** = [3, 4, 5, 6, 7, 8]
	weights = [‘uniform’, ‘distance’]	**gamma** = [0.001, 0.01, 0.05, 0.1, 0.2, 0.5, 1, 10, 100]	**learning_rate** = [0.001, 0.01, 0.05, 0.1, 0.2, 0.3]
	p = [1, 2]		**n_estimators** = [50, 100, 150, 200, 250]
			subsample = [0.6, 0.8, 1.0]
			colsample_bytree = [0.6, 0.8, 1.0]

And the winner is...

Models were checked in terms of balanced accuracy, precision and recall (for 0 and 1). 2 scores were checked: 1. performing on cross validation itself, and 2. performing on validation sample.

On cross validation the best scores for each of the components were achieved by **SVM model with C = 5, gamma - 0.1 and Random Under Sampler** - resulting with **balanced accuracy = 0.7551**

However it was not consistent with check on validation sample, as the best scores for this prediction were shown using also **SVM model** but with different parameters and different resampling method: **C = 1, gamma = 0.01 and sampler SMOTETomek** - resulting with **balanced accuracy = 0.7858**

Because of resampling we decided to conduct prediction on test sample using the method with the best scores in predicting validation sample : **SVM, C = 1, gamma = 0.01 and sampler SMOTETomek**